

Part 2 — Demo Script (Loom follow-along)

Target ~8-10 min. `[SHOW ...]` = what to put on screen (exact file + line numbers given). Plain text = what you say.

Open these tabs before recording:

- Live Vercel app (`/` , `/register` , `/my` , `/staff`)
- Repo in your editor
- `keiki-ERD.png` (the rendered diagram) — for the data-model section
- Stripe Dashboard → Developers → Webhooks → your endpoint
- A terminal in the project root, ready to run `npm run test:concurrency`

Line numbers below match the committed code. If they drift after edits, the symbol names in **bold** still locate the spot.

0. Intro (20s)

[SHOW live Vercel app home page]

"This is the parent registration system I built. Parents register their kids for after-school classes and pay at signup. I'll cover the tools, the data model, and then the specific edge cases you asked about — the exact creation order, failure handling, and the oversell problem."

1. Tools used (40s)

[SHOW `README.md`]

"Stack: **Next.js** App Router + TypeScript on **Vercel**, **Supabase Postgres** for the database, **Stripe Checkout** for payment. Database access is raw SQL through node-postgres — no ORM — so the two queries that carry the real weight, the atomic seat claim and the guarded release, stay visible instead of hidden behind a query builder.

The one piece off my usual stack is **n8n** — I used it for the confirmation email. Deliberate: it's the tool you run day-to-day, so I wanted to show I can pick it up, and I kept it to the low-risk side effect — email — with the transactional core in Postgres where it belongs. I also used AI as a build assistant."

2. Data model + why (90s)

[SHOW `keiki-ERD.png` — the rendered diagram] (full write-up + DDL in `keiki-ERD.md`)

"Nine tables. A few decisions worth calling out — all in the ERD doc:

- **School → Class → Session.** A Class holds the recurrence — day, time, start date, weeks. But **Session is its own table**, one row per meeting, because you said sessions get canceled for holidays or rescheduled. Computing them from the rule would leave nowhere to record 'week 4 is canceled.'
Rationale: `keiki-ERD.md §3.1` .
- **Parent → Child → Enrollment.** Enrollment is the join between a child and a class — a parent has many children, a child is in many classes, that falls out of the join. It carries the lifecycle status plus `join_date / drop_date` , which is how mid-semester joins and drops are represented.
- **Payment funds many enrollments** — one checkout can pay for several child-plus-class rows (`keiki-ERD.md §3.2`).
- **Submission** is the idempotency anchor — I'll come back to it.

Two things I made the schema *ready* for without building: mid-semester **proration** (full price in v1, but `join_date` is there — `§3.5`), and **cancellation as a status transition, never a delete** (`§3.4`), so history is never lost."

3. The registration flow — exact creation order (2.5 min)

[SHOW `/register` on the live app; fill parent + one child + tick one class]

"Let me register a child, then walk through exactly what gets created, in order. The whole flow lives in one service — `lib/registration/register.ts` , function `registerSubmission` (line 56)."

[SHOW `lib/registration/register.ts:56` while you narrate the steps]

"On submit, in this order:

Step 1 — Upsert the parent. `register.ts:66` calls `upsertParent` in `lib/registration/parent.ts:21` . That's an `INSERT ... ON CONFLICT (email) DO UPDATE ... RETURNING id` — `parent.ts:25` . Email is the natural key, case-insensitive (citext). **This is how I know if the parent already exists:** a hit on email reuses the row, a miss creates it. And it's race-safe — two simultaneous first-time submissions with the same email both resolve to one parent, because the unique constraint drives the upsert instead of a check-then-insert.

Step 2 — Idempotency gate. `register.ts:69` calls `insertSubmission` (`lib/registration/submission.ts:22`), which does `INSERT ... ON CONFLICT (id) DO NOTHING` on the client-generated submission UUID (`submission.ts:31`). If the row already exists, we branch to `resumeExisting` (`register.ts:71`) instead of charging again. I'll show why in section 4.

Step 3 — One database transaction (`register.ts:126` , `withTransaction`). Inside it, in order:

- create/find the children — `findOrCreateChildren ( register.ts:127 )`
- **atomic seat claim per class** — the loop at `register.ts:141` ; if any claim fails the whole transaction rolls back (that's the all-or-nothing guarantee)
- insert one pending **Payment** (`register.ts:152`) and the pending **Enrollment** rows (`register.ts:160`), with `payment_id` linked at creation so the webhook can find them
- link the submission to the payment (`register.ts:166`)

Step 4 — After the transaction commits, create the Stripe Checkout session (`register.ts:201`), store its URL, mark the submission `checkout_created` , and redirect."

[On the app, CLICK Register & Pay → lands on Stripe Checkout]

"**Two kids in one submission?** [note you could add a second child on the form] It's still **one** checkout and **one** payment funding **N** enrollment rows — and it's all inside that single transaction (`register.ts:126`), so if any one child's seat claim fails, nothing is created and no charge starts. Atomic at the database level, not just the happy path."

[Pay with test card `4242 4242 4242 4242`]

"**Now the important one — payment succeeds but record creation fails halfway.** The browser redirect does **not** finalize anything. The **Stripe webhook is the source of truth** — `app/api/webhook/route.ts` ."

[SHOW `app/api/webhook/route.ts:31` (the `checkout.session.completed` case) and `lib/registration/finalize.ts:17` (`finalizeCheckout`)]

"The pending rows were written *before* Stripe (step 3). When payment completes, Stripe calls my webhook (`route.ts:31`), which calls `finalizeCheckout` (`finalize.ts:17`) to flip the rows to active. So if the user closes the tab right after paying, the registration still completes when the webhook lands. And finalize is **idempotent** — it locks the payment row `FOR UPDATE` (`finalize.ts:24`) and short-circuits if it's already `paid` (`finalize.ts:28`) — so a retried or duplicated webhook can't double-register or double-count a seat."

[SHOW `/my` — the registration now shows **Active**]

"There's the confirmed registration."

4. Duplicate submission (40s)

[SHOW `app/register/page.tsx:72` — `crypto.randomUUID()` generated once; then `submission.ts:31` — the `ON CONFLICT DO NOTHING`]

"**Same submission twice** — double-click, or hit back and resubmit. The form generates the submission UUID **once** (`page.tsx:72`) and sends it every time. On the server, `INSERT ... ON CONFLICT (id) DO NOTHING` (`submission.ts:31`): the second attempt finds the existing row and returns the checkout that already

exists instead of charging again. Resubmit is a no-op, not a second charge. There's a test that asserts exactly this — `tests/idempotency.test.ts` : same UUID twice, one row, one payment."

5. Oversell — 50 parents, 12 seats (90s)

[SHOW `lib/registration/claimSeat.ts:12` — the `claimSeat` function / `UPDATE class ... WHERE seats_taken < capacity` at lines 14-18] (also `keiki-ERD.md §4`)

"The one you really asked about. 8am, 50 parents, a 12-seat class. The seat claim is a single atomic statement (`claimSeat.ts:14`):

```
UPDATE class SET seats_taken = seats_taken + 1 WHERE seats_taken < capacity RETURNING seats_taken
```

Postgres serializes the row-level write, so of 50 concurrent requests exactly 12 get a row back and 38 get `null` — and the 38 fail closed, rolling back the whole submission. No lock is held across the slow Stripe call; the claim is one statement. Seats are also reserved *before* Stripe with a 35-minute hold that outlives the 30-minute checkout window, so a slow payer can't pay into a released seat (`keiki-ERD.md §3.6`)."

[SHOW terminal → run `npm run test:concurrency`]

"Here's the proof — the test fires 50 concurrent claims at the 12-seat class (`tests/concurrency.test.ts:22`)."

[SHOW output line: `attempts=50 ... winners=12 losers=38 seats_taken=12` → `PASS`]

"Exactly 12, every time. And this isn't the first time I've shipped this pattern — in DermaRoute I used the same atomic-counter approach, `INSERT ... ON CONFLICT DO UPDATE ... RETURNING` on a shared Postgres counter, to cap daily API spend: reserved before each billable call, fail-closed, correct across concurrent serverless instances. Structurally the same problem as not overselling a class."

6. What I'd deliberately leave out of v1 (45s)

[SHOW `workflow/PRD.md:135` — the "Out of Scope (Future)" list]

"Scope I cut on purpose, and why:

- **Refund automation** — cancellations are staff-approved and refunded manually via the Stripe dashboard. Automating refund policy is real product work, not core plumbing.
- **Waitlists** — the schema wouldn't fight it, but it's a feature, not a foundation.
- **Prorated mid-semester billing** — full price in v1, schema already carries `join_date`.
- **Multi-currency, SMS reminders, a full admin dashboard, and real user accounts** — v1 auth is a minimal signed-cookie lookup.

The point wasn't to build everything — it was to get the transactional core right and cut the rest cleanly."

7. What I'd need to know about your real system (45s)

[SHOW workflow/PRD.md:116 — §5.8]

"Before building this for real, my questions are about how it fits your existing Airtable and n8n setup:

- Does this **replace** Fillout-plus-Airtable for registration, or **feed** Airtable as a new front door?
- Once this exists, **who owns the class roster** — this database or Airtable? There has to be one source of truth.
- **How does payroll consume this?** It runs through n8n and Airtable today — does it need enrollment or attendance data out of this system, and in what shape?
- Are there existing student or parent IDs I'd need to reconcile against on import?

Those answers change the integration boundary, so I'd want them before committing to a design."

8. Close (15s)

"So: the data model built around the tricky cases, the webhook as the source of truth for reliable payments, and an atomic seat claim that can't oversell. Code, tests, and design docs are all in the repo. Thanks for taking a look."

Quick reference — where each answer lives

Question	Say it with	File : line
Tools used	README	<code>README.md</code>
Data model + why	ERD image + rationale	<code>keiki-ERD.png</code> , <code>keiki-ERD.md §3</code>
Creation order	the service	<code>lib/registration/register.ts:56</code> (steps at 66, 69, 126, 141, 152, 160, 201)
Parent already exists?	email upsert	<code>lib/registration/parent.ts:21</code> (ON CONFLICT :25)
Two kids one submission	one txn, N enrollments	<code>lib/registration/register.ts:126,160</code>
Payment succeeds, record fails	webhook = truth, idempotent	<code>app/api/webhook/route.ts:31</code> → <code>lib/registration/finalize.ts:17</code> (FOR UPDATE :24 , paid short-circuit :28)
Same submission twice	client UUID gate	<code>app/register/page.tsx:72</code> → <code>lib/registration/submission.ts:31</code>
Oversell 50/12	atomic claim + live test	<code>lib/registration/claimSeat.ts:14</code> , <code>tests/concurrency.test.ts:22</code>
Leave out of v1	out-of-scope list	<code>workflow/PRD.md:135</code>
Need to know about their system	integration questions	<code>workflow/PRD.md:116</code>

Pre-record checklist

- ☐ [] Live Vercel app loads; classes show seats remaining.
- ☐ [] Stripe in **test mode**; dashboard webhook endpoint shows recent `200` s.
- ☐ [] `npm run test:concurrency` runs green in a ready terminal.
- ☐ [] Use a **non-personal** test email on the form (not your account email).
- ☐ [] `keiki-ERD.png` , `keiki-ERD.md` , `workflow/PRD.md` , `README.md` open in tabs.
- ☐ [] Wise email / phone ready to include with the Loom link (per the brief).